LLMs and Tool Use

Super AI Engineer Level 2

Nutchanon Yongsatianchot (Nut)

CHA

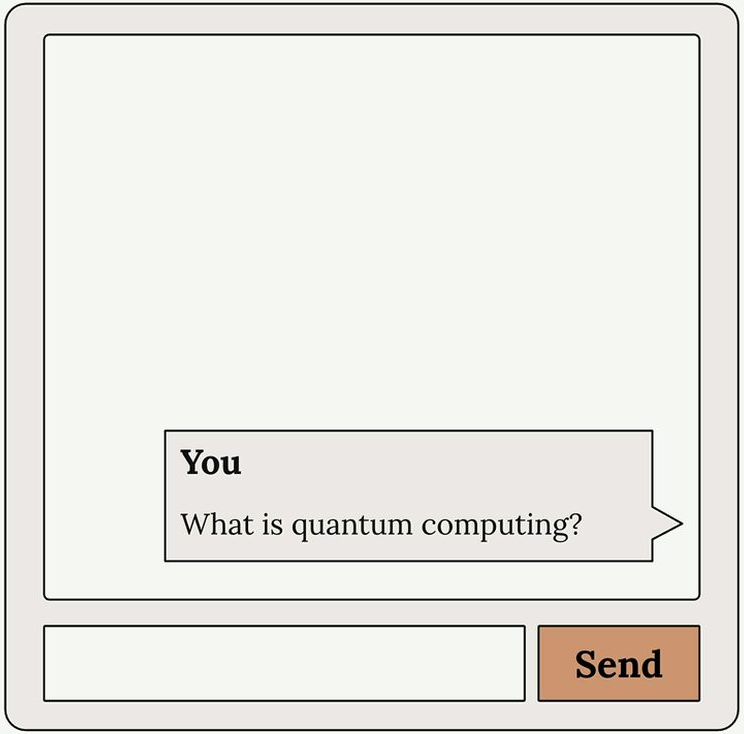
10th ANIVERSARY

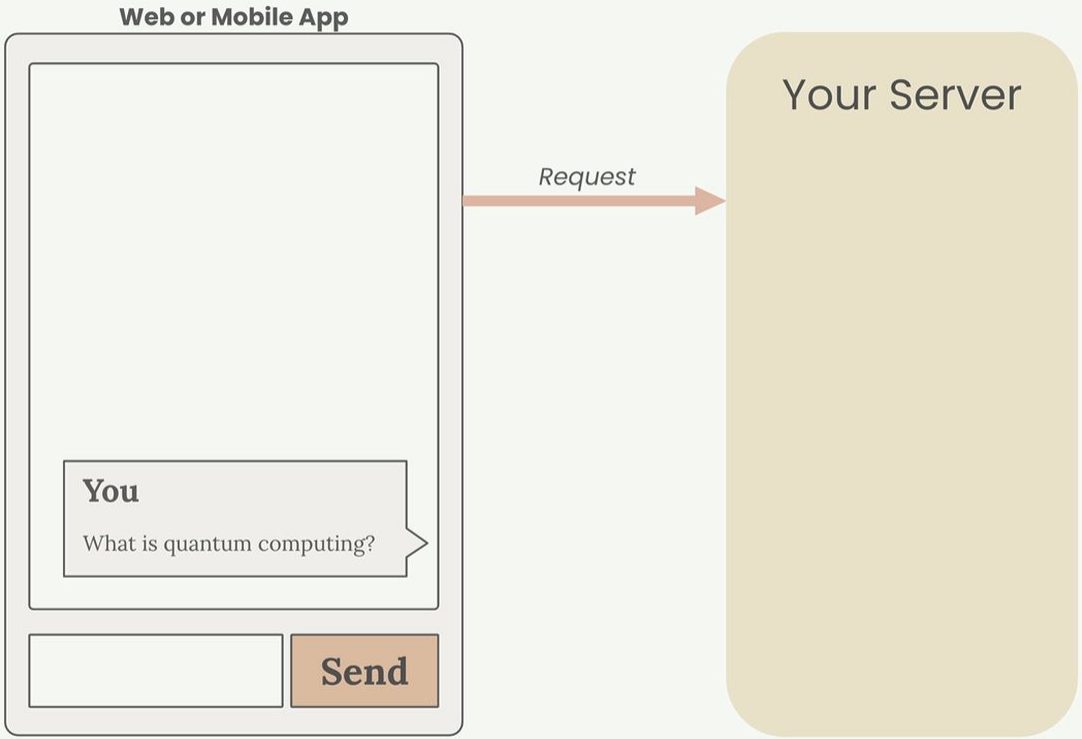
The LLM API Request Flow

Diagrams from [Claude with the Anthropic API]

- **The LLM API Request Flow**
- API Demo
- Prompt Engineering
- Tool Use

Chat example





Requests to Anthropic should not be made directly from client code

API requests requires a secret API key, which shouldn't be exposed to the client

Requests to the API can be made through a SDK or with a plain HTTP request

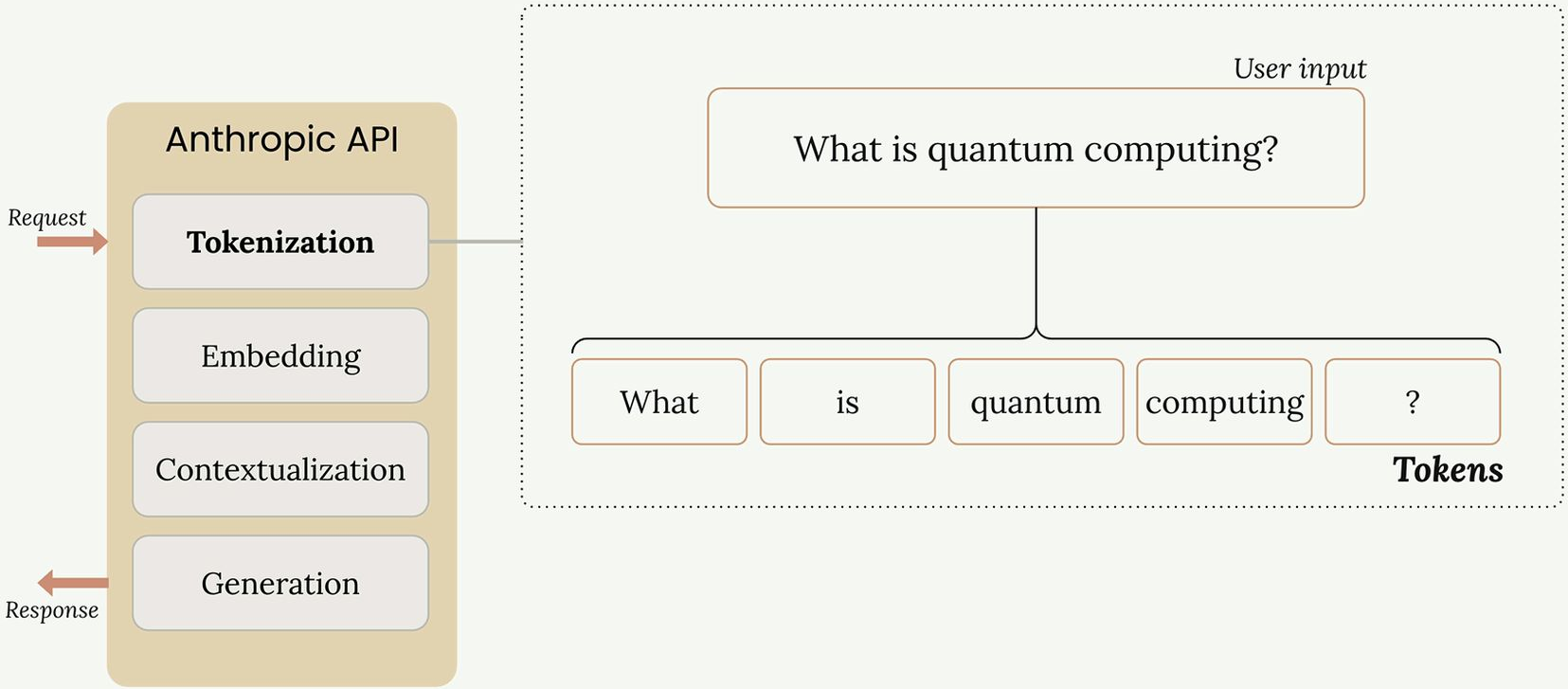
Input text is placed inside of a "user" message, which is then placed in a list of messages

Your Server

Anthropic SDK

Field	Purpose
API Key	Identifies your request to Anthropic
Model	Name of the model to use
Messages	List of "messages" containing the user's input text
Max Tokens	Limit to how many "tokens" the model can generate

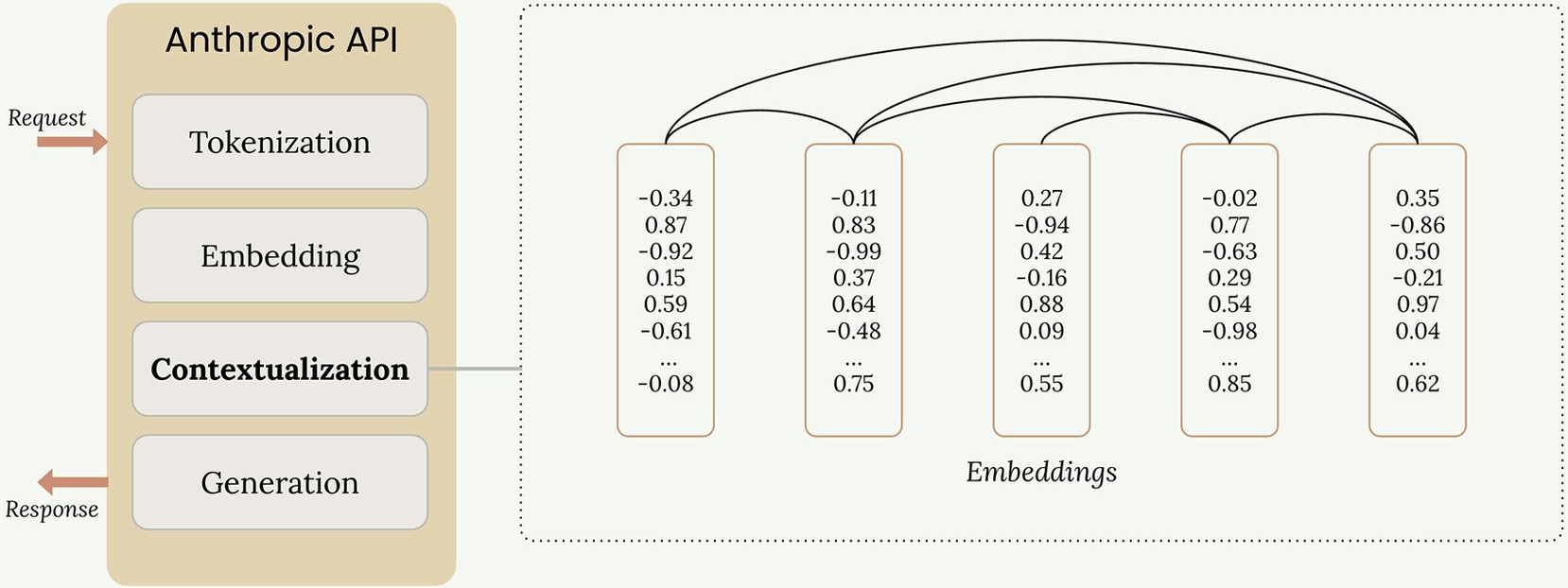
Anthropic API

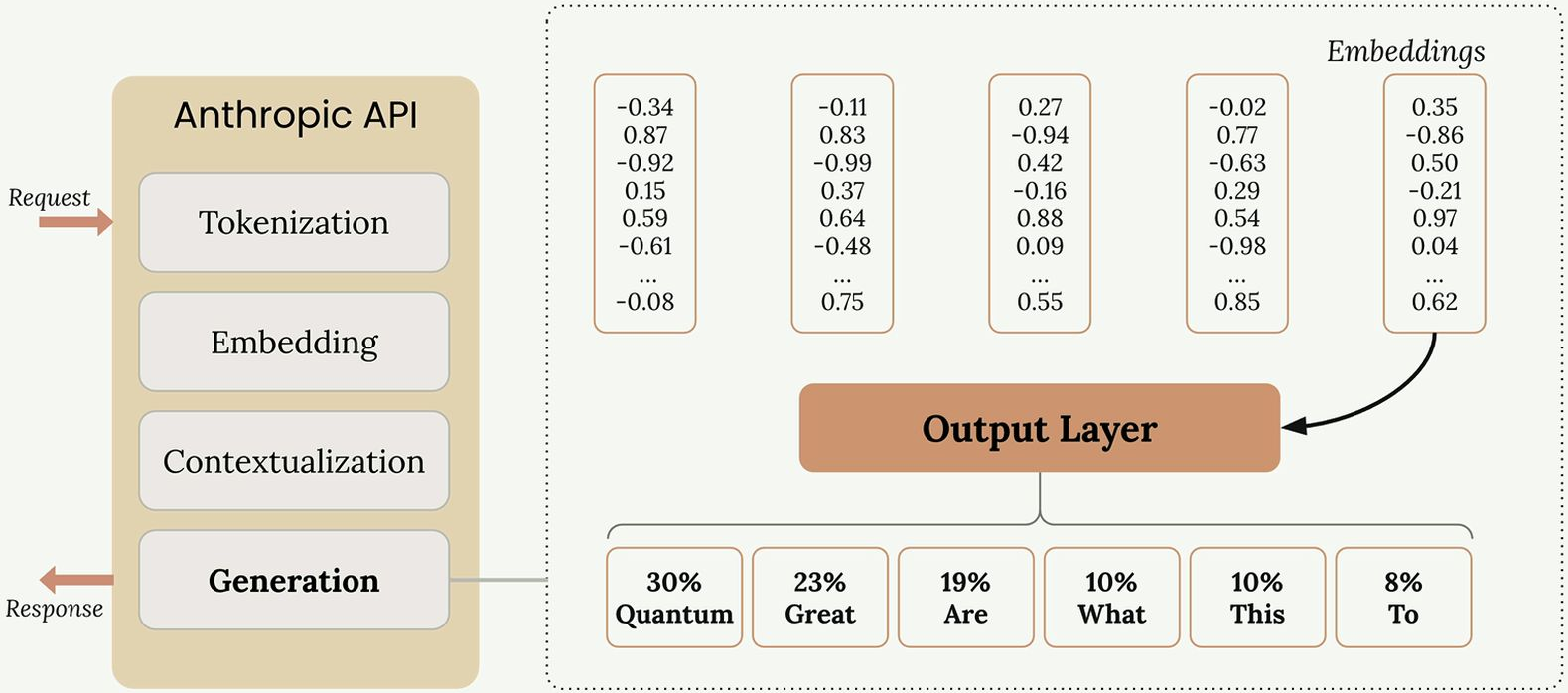

Anthropic API
Request
Tokenization
Embedding
Contextualization
Generation
Response
User input
What is quantum computing?
What
is
quantum
computing
?
Tokens

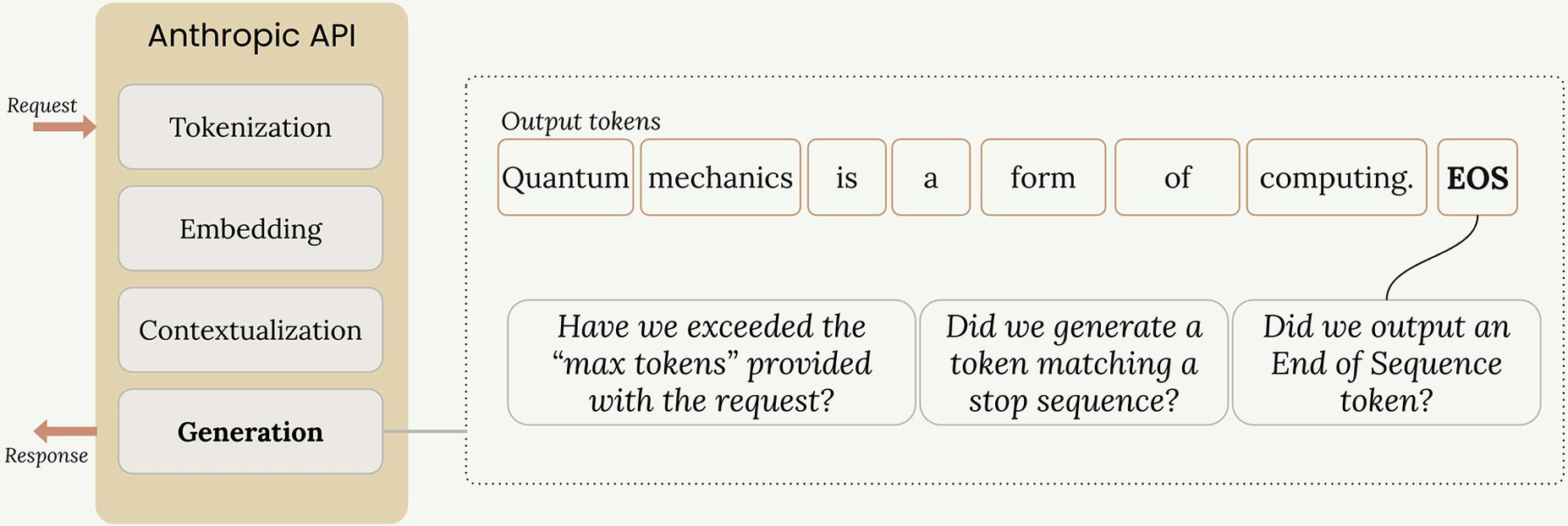
Possible meanings of “quantum”

quantum

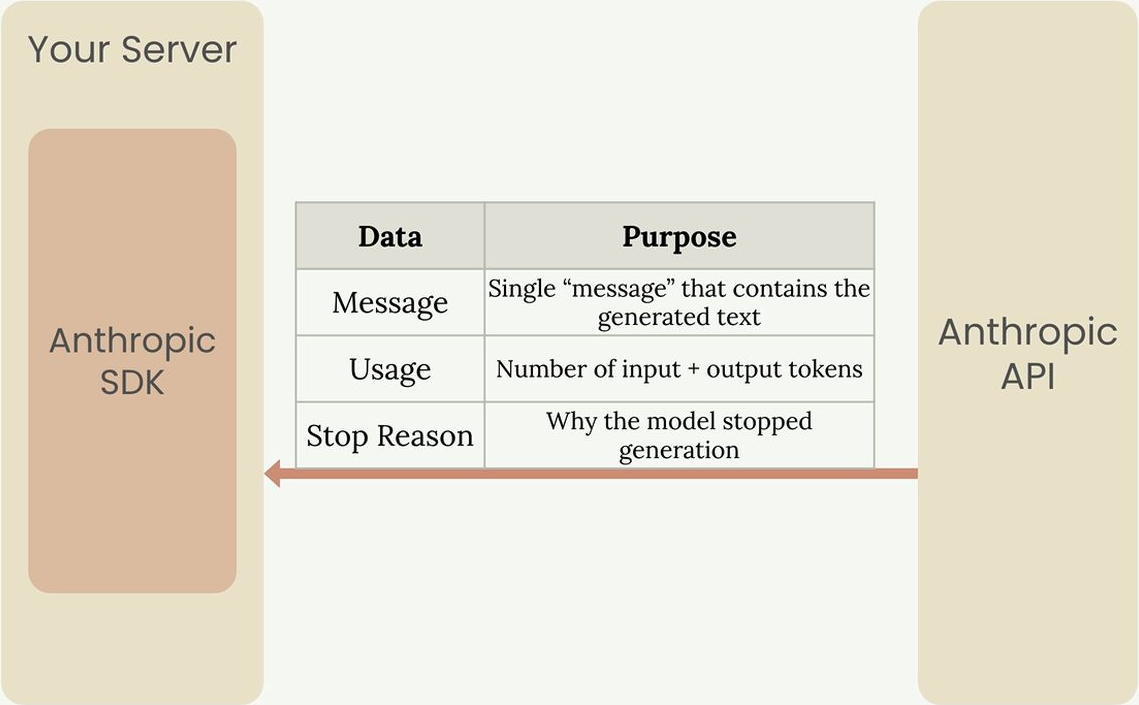
- A discrete, indivisible unit of a physical quantity, especially energy. From Latin "quantus" meaning "how much."
- Relating to quantum mechanics or quantum physics, a branch of physics dealing with atomic and subatomic particles.
- Extremely small or occurring at the subatomic level.
- Relating to quantum computing, which uses quantum-mechanical phenomena to perform operations on data.


Anthropic API
Request
Tokenization
Embedding
Contextualization
Generation
Response
-0.34
0.87
-0.92
0.15
0.59
-0.61
...
-0.08
-0.11
0.83
-0.99
0.37
0.64
-0.48
...
0.75
0.27
-0.94
0.42
-0.16
0.88
0.09
...
0.55
-0.02
0.77
-0.63
0.29
0.54
-0.98
...
0.85
0.35
-0.86
0.50
-0.21
0.97
0.04
...
0.62
Embeddings

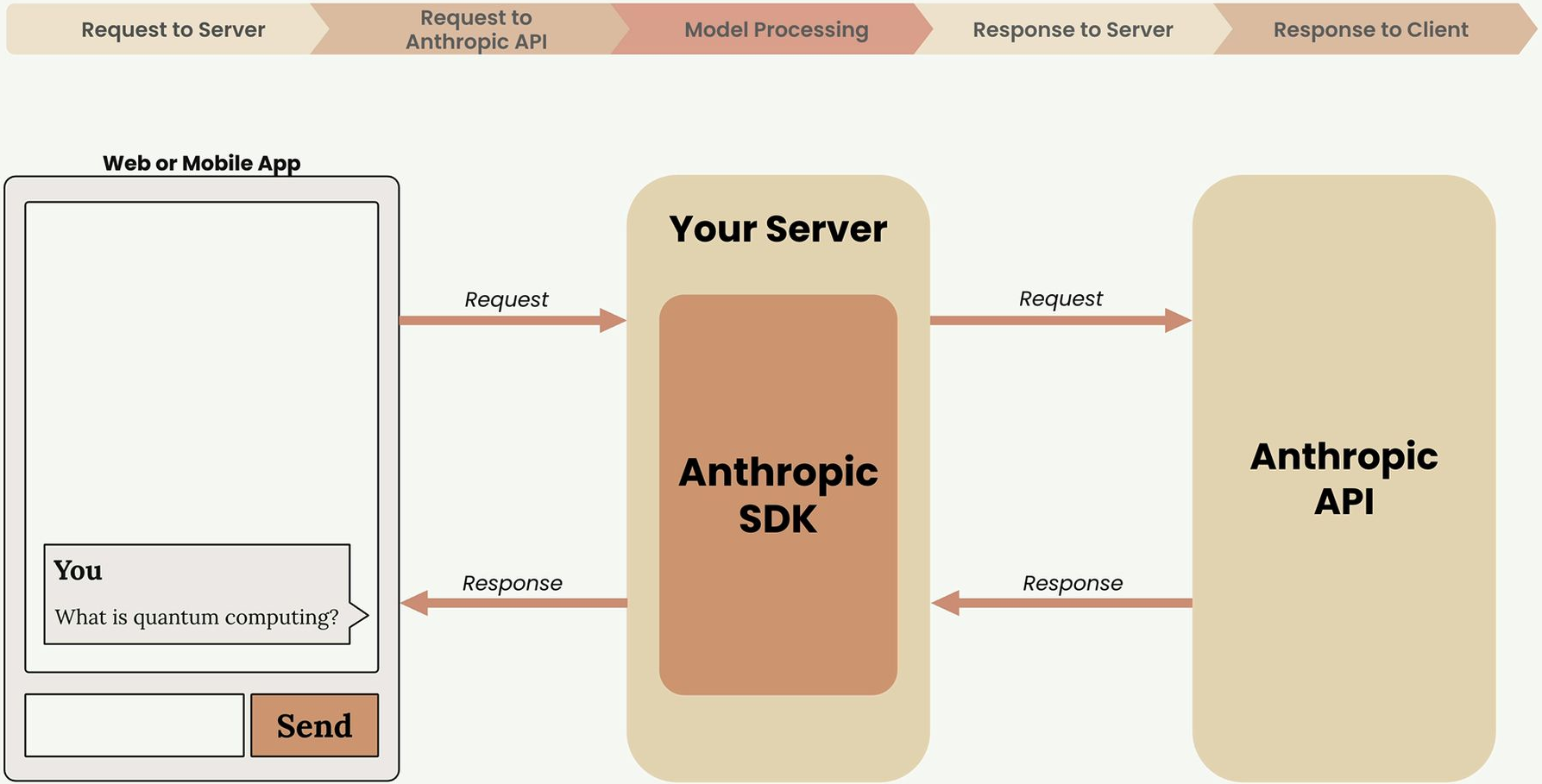

Anthropic API
Request
Tokenization
Embedding
Contextualization
Generation
Response
Embeddings
-0.34 0.87 -0.92 0.15 0.59 -0.61 ... -0.08
-0.11 0.83 -0.99 0.37 0.64 -0.48 ... 0.75
0.27 -0.94 0.42 -0.16 0.88 0.09 ... 0.55
-0.02 0.77 -0.63 0.29 0.54 -0.98 ... 0.85
0.35 -0.86 0.50 -0.21 0.97 0.04 ... 0.62
Output Layer
30% Quantum
23% Great
19% Are
10% What
10% This
8% To


Anthropic API
Request
Tokenization
Embedding
Contextualization
Generation
Response
Output tokens
Quantum
mechanics
is
a
form
of
computing.
EOS
Have we exceeded the "max tokens" provided with the request?
Did we generate a token matching a stop sequence?
Did we output an End of Sequence token?

Output text placed into an "assistant" message



Data	Purpose
Message	Single "message" that contains the generated text
Usage	Number of input + output tokens
Stop Reason	Why the model stopped generation


Request to Server
Request to Anthropic API
Model Processing
Response to Server
Response to Client
Web or Mobile App
You
What is quantum computing?
Send
Request
Response
Your Server
Anthropic SDK
Request
Response
Anthropic API

API Demo

- The LLM API Request Flow
- **API Demo**
- Prompt Engineering
- Tool Use

Message structure

- message = a list of interaction between ‘user’ and ‘assistant’
  - Conversation history
- messages = [{'role':'user', 'content':'Hello'},

  {'role':'assistant','content':'Hello, how can I help you?'},

  {'role':'user','content':'Can you write me an email?'},

  {'role','assistant','content':'Sure. What is it about?'}]

Stateless Conversation:
The model does not store any message

How Multi-Turn Conversations Work

To maintain conversation context, you need to do two things:

- Manually maintain a list of all messages in your code
- Send the complete message history with every request

Flow:

1. Send your initial user message to LLM
2. Take LLM's response and add it to your message list as an assistant message
3. Add your follow-up question as another user message
4. Send the entire conversation history to LLM

System Message

- A system prompt is a way to **provide context, instructions, role, personality, tone, and guidelines** to the model before presenting it with a question or task.

API & Multi-turn Conversation Demo

https://colab.research.google.com/drive/1dxPCd5kJK4hhnDxuhYYReiobAD2DjBSp?usp=sharing

Prompt Engineering

- The LLM API Request Flow
- API Demo
- **Prompt Engineering**
- Tool Use

What is Prompt Engineering?

Prompt Engineering = “A process of creating an instruction you provide to the model to do a particular task well.”

Rule of thumb: “Think of LLMs as a new coworker just joining a project.”

Everyday Prompting vs. Production Prompting

Consumer prompts

- More **open-ended**, **less structured**
- Meant for **one-off tasks**
- Conversational, exploratory
- On the **shorter** side

Production prompts

- Meant for **high-throughput, repetitive, or scaled** tasks
- Need to test carefully
- **Highly structured**
- On the **longer** side

The Core Prompting Techniques

- Provide Context
- Be specific about output
- Break down complex tasks into steps
- Provide examples
- Define role or persona

Provide Context

Before

Tell me about climate change

After

I need to explain climate change to my five year old son. Provide explanation suitable for my son.

After

Explain three major impacts of climate change on agriculture in Thailand, with examples from the past decade.

Think about what a new teammate needs to know to do the task such as objectives, audience and other relevant informations.

Be specific about output

Before

Write a short message

After

Total reply should be between 300 - 500 words

Be specific about what you want, providing details about scope, requirements, formatting, and other relevant parameters.

Be specific about output

Before

Design me a personal art portfolio website.

After

Build a minimalist art portfolio with a fullscreen hero, filterable masonry gallery (by medium: digital, oil, watercolor), artist statement, exhibition history, and contact form.

Use a clean gallery-white aesthetic with subtle hover animations and lazy-loading for images.

Be specific about what you want, providing details about scope, requirements, formatting, and other relevant parameters.

Break down complex tasks into steps

Before

Analyze this quarterly sales data.

After

Analyze this quarterly sales data with the following approach:

1. Break down performance by region (strongest vs. weakest markets)
2. Calculate year-over-year growth rates
3. Flag any products at risk of declining
4. Provide 2-3 actionable recommendations for next quarter

Breaking down complex tasks into clear steps guides that you want AI to do.

Provide examples

Before

Summarize this article in a professional tone.

After

Summarize this article in a professional tone. Here are examples of the style I'm looking for:

- **Good**: "The study found a 23% increase in productivity among remote workers, suggesting potential benefits for hybrid workplace models."
- **Good**: "Market analysts attribute the decline to supply chain disruptions and shifting consumer preferences."
- **Avoid**: "So basically the study says working from home is way better and companies should totally do it!"

Now, please summarize this article using that professional tone.

Providing (good and bad) examples helps the AI understand the pattern, style, or format you're looking for more clearly than descriptions alone.

Define role or persona

Before

Explain how rainbows form.

After

You are an experienced science teacher. Explain how rainbows form as if you are speaking to a bright 10-year-old who's interested in science.

Defining the AI's role, tone, or style helps shape its approach to fit your specific needs and audience.

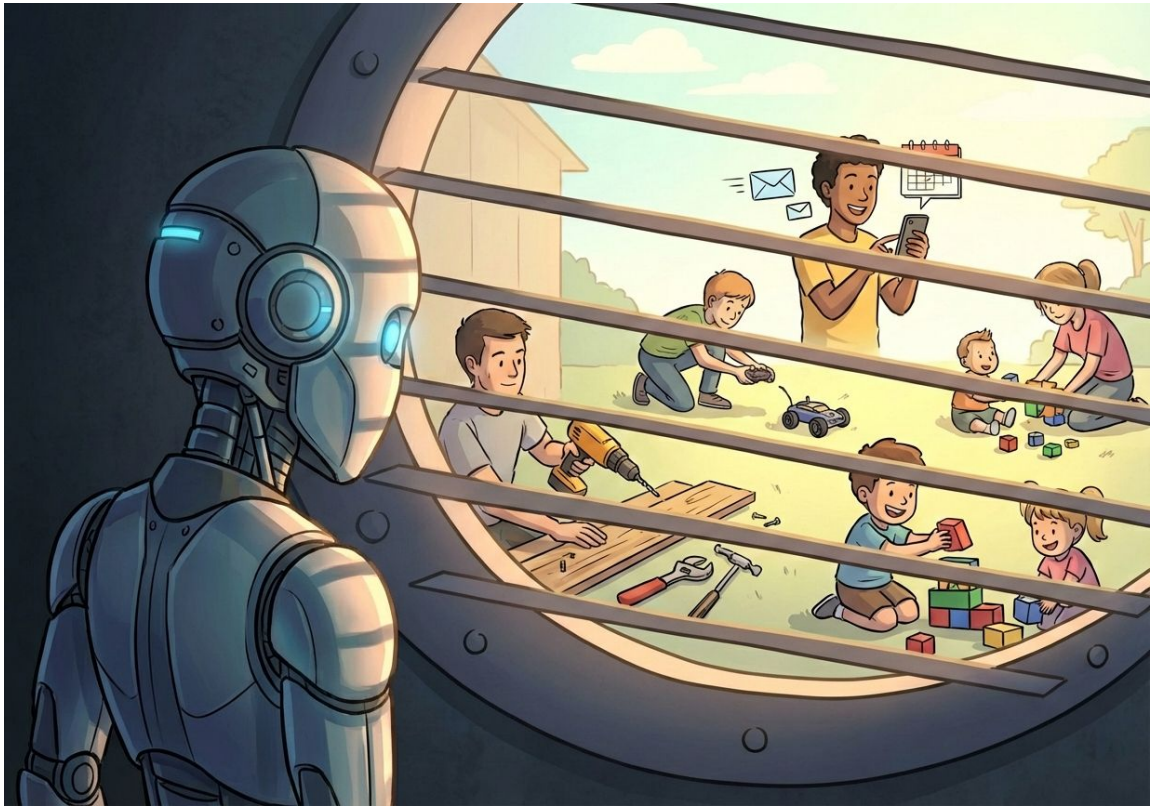
Ask the AI for help with prompting

- When you're not sure how to ask for something, the AI can help improve your prompt.
- You can talk to AI to see how they understand your prompt too!
- **Important note**: Only you know the requirements and the context of your task. Do not offload everything to AI

Tool Use

- The LLM API Request Flow
- API Demo
- Prompt Engineering
- **Tool Use**

How can LLMs interact with the world?

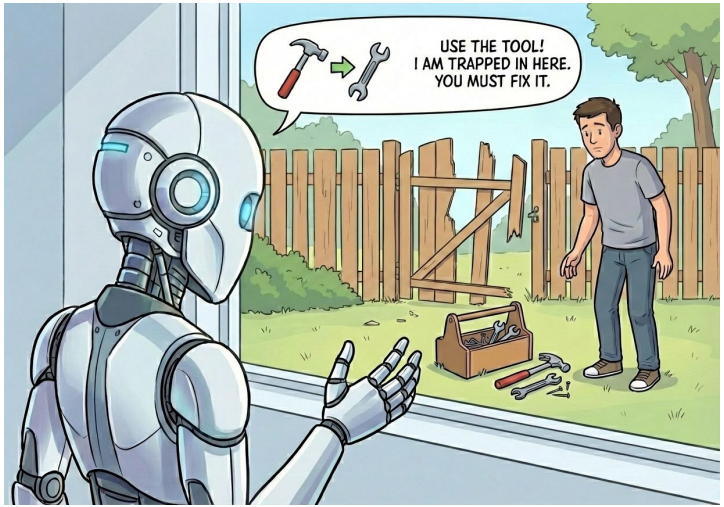


CHA

Key Benefits

- **Real-time Information**: Access current data that wasn't available during LLM's training
- **External System Integration**: Connect LLM to databases, APIs, and other services
- **Structured Interaction**: Allow LLM to interact with your computer

How can LLM use tools?



Answer: LLM tells “us” to do it for them.

“App” → Call a function

Tool Use

- Provide tools to LLMs and they can decide which tools to use and when.

Q: You need to tell the current temperature in Bangkok. Which tool would you use?

A) `get_weather(city)`

B) `get_current_time(city)`

C) `get_calendar(date)`

D) simply say “Hello”

CHA

Tool Use

- Provide tools to LLMs and they can decide which tools to use and when.

Q: You need to tell the current temperature in Bangkok. Which tool would you use?

A) `get_weather(city)`

B) `get_current_time(city)`

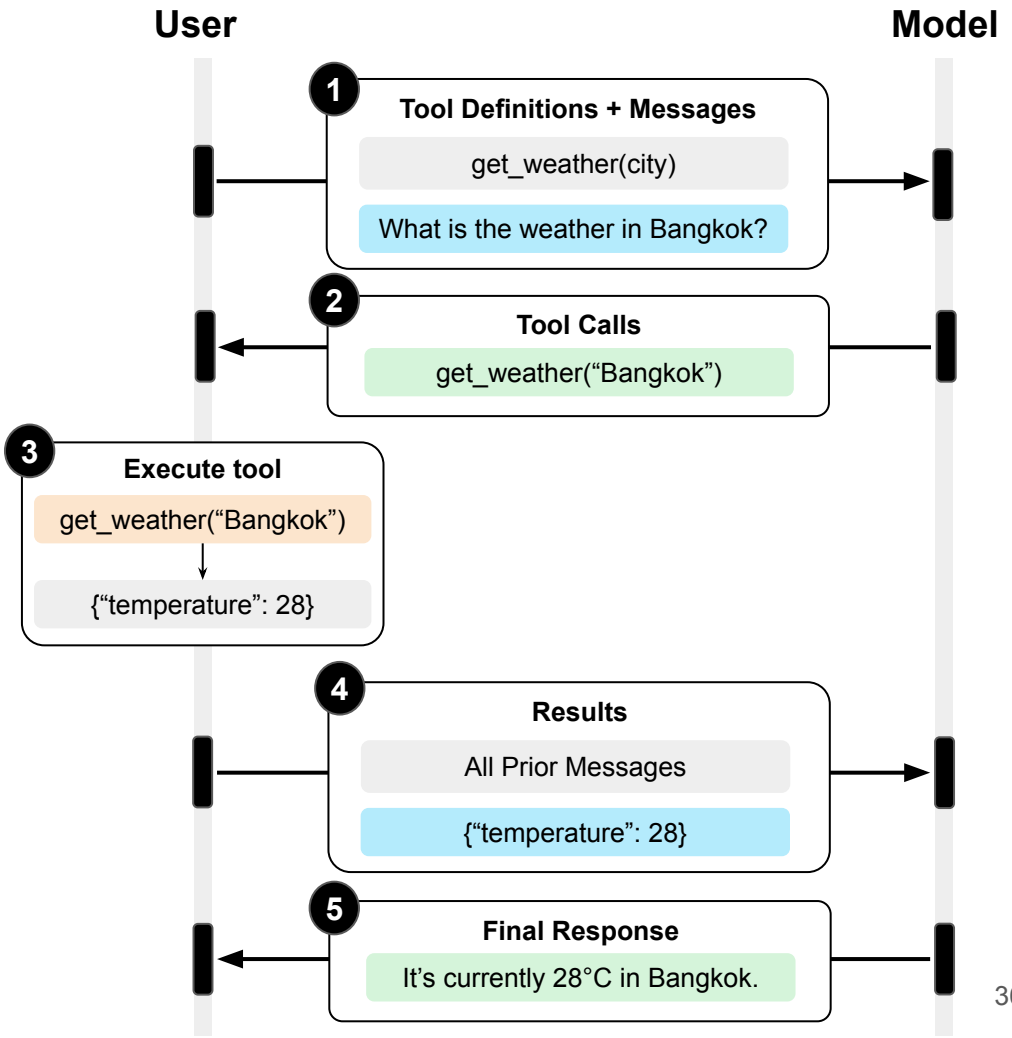
C) `get_location(city)`

D) `get_calendar(date)`

→ `get_weather("BKK")`

Tool Use

- Provide tools to LLMs and they can decide what tools to use and when.
- A tool (function) is called separately and the outputs are then appended back to the conversation.



How tool use works

1. **Provide LLM with tools and a user prompt**
   - Define tools with **names**, **descriptions**, and **input schemas** in your API request.
   - Include a user prompt that might require these tools.
     - e.g., “What’s the weather in San Francisco?”
2. **LLM decides to use a tool**
   - LLM assesses if any tools can help with the user’s query.
   - If yes, **LLM constructs a properly formatted tool use request**.
   - The API response has a `stop_reason` of `tool_use`, signaling LLM’s intent.
3. **Extract tool input, run code, and return results**
   - **On your end**, extract the tool name and input from LLM’s request.
   - Execute the actual tool code client-side.
   - Continue the conversation with **a new** `user` **message** containing a `tool_result` content block.
4. **LLM uses tool result to formulate a response**
   - LLM analyzes the tool results to craft its final response to the original user prompt.

All tools are user-provided

Specifying tools (OpenAI Style)

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "A simple calculator that performs basic arithmetic operations.",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "The mathematical expression to evaluate (e.g., '2 + 3 * 4')."
                    }
                },
                "required": ["expression"]
            }
      }
    }
```

https://openrouter.ai/docs/features/too

Specifying tools (Anthropic Style)

```
{
  "name": "get_weather",
  "description": "Get the current weather in a given location",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "The city and state, e.g. San Francisco, CA"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "The unit of temperature, either 'celsius' or 'fahrenh
      }
    },
    "required": ["location"]
  }
}
```

CHA

10th ANIVERSARY

Best practices for tool definitions

- **Provide extremely detailed descriptions.** Your descriptions should explain every detail about the tool, including:
  - What the tool does
  - When it should be used (and when it shouldn't)
  - What each parameter means and how it affects the tool's behavior
  - Any important caveats or limitations, such as what information the tool does not return if the tool name is unclear.
- **Prioritize descriptions over examples.**

Example of a poor tool description

```
{
  "name": "get_stock_price",
  "description": "Gets the stock price for a ticker.",
  "input_schema": {
    "type": "object",
    "properties": {
      "ticker": {
        "type": "string"
      }
    },
    "required": ["ticker"]
  }
}
```

Example of a good tool description

```
{
  "name": "get_stock_price",

  "description": "Retrieves the current stock price for a given ticker symbol. The ticker symbol must be a valid symbol for a publicly traded company on a major US stock exchange like NYSE or NASDAQ. The tool will return the latest trade price in USD. It should be used when the user asks about the current or most recent price of a specific stock. It will not provide any other information about the stock or company.",

  "input_schema": {
    "type": "object",
    "properties": {

      "ticker": {
        "type": "string",
        "description": "The stock ticker symbol, e.g. AAPL for Apple Inc."
      }
    },
    "required": ["ticker"]
  }
}
```

Handling tool use and tool result content blocks

- When LLM decides to use one of the tools you've provided, **it will return a response with a `stop_reason` of `tool_use`** and one or more `tool_use` **content blocks** in the API response that include:
  - `id`: A unique identifier for this particular tool use block. This will be used to match up the tool results later.
  - `name`: The name of the tool being used.
  - `input`: An object containing the input being passed to the tool, conforming to the tool's `input_schema`.

```
{
  "id": "msg_01Aq9w938a90dw8q",
  "model": "claude-3-5-sonnet-20240620",
  "stop_reason": "tool_use",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "<thinking>I need to use the get_weather, and the user wants SF
    },
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lq9",
      "name": "get_weather",
      "input": {"location": "San Francisco, CA", "unit": "celsius"}
    }
  ]
}
```


CHA

10th ANIVERSARY

Things to do with a tool use response

1. Extract the `name`, `id`, and `input` from the `tool_use` block.
2. Run the actual tool in your codebase corresponding to that tool name, passing in the tool `input`.
3. [optional] Continue the conversation by sending a new message with the `role` of `user`, and a content block containing the `tool_result` type and the following information:
   - `tool_use_id`: The `id` of the tool use request this is a result for.
   - `content`: The result of the tool, as a string (e.g. `"content": "15 degrees"`) or list of nested content blocks (e.g. `"content": [{"type": "text", "text": "15 degrees"}]`). These content blocks can use the `text` or `image` types.
   - `is_error` (optional): Set to `true` if the tool execution resulted in an error.

Example of successful tool results

JSON

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
      "content": "15 degrees"
    }
  ]
}
```

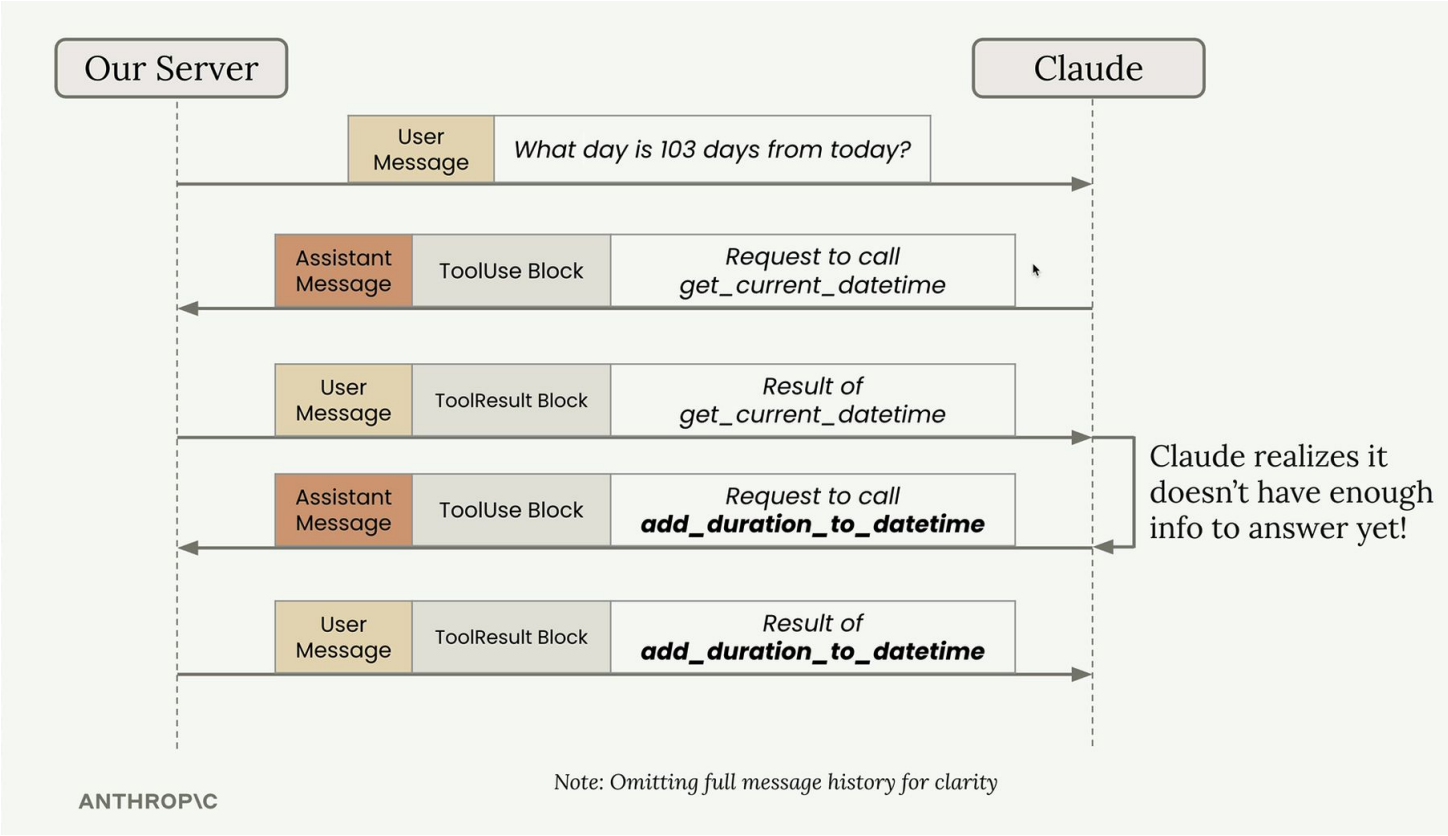
CHA

10th ANIVERSARY

Tool Demo

https://colab.research.google.com/drive/1dxPCd5kJK4hhnDxuhYYReiobAD2DjBSp?usp=sharing

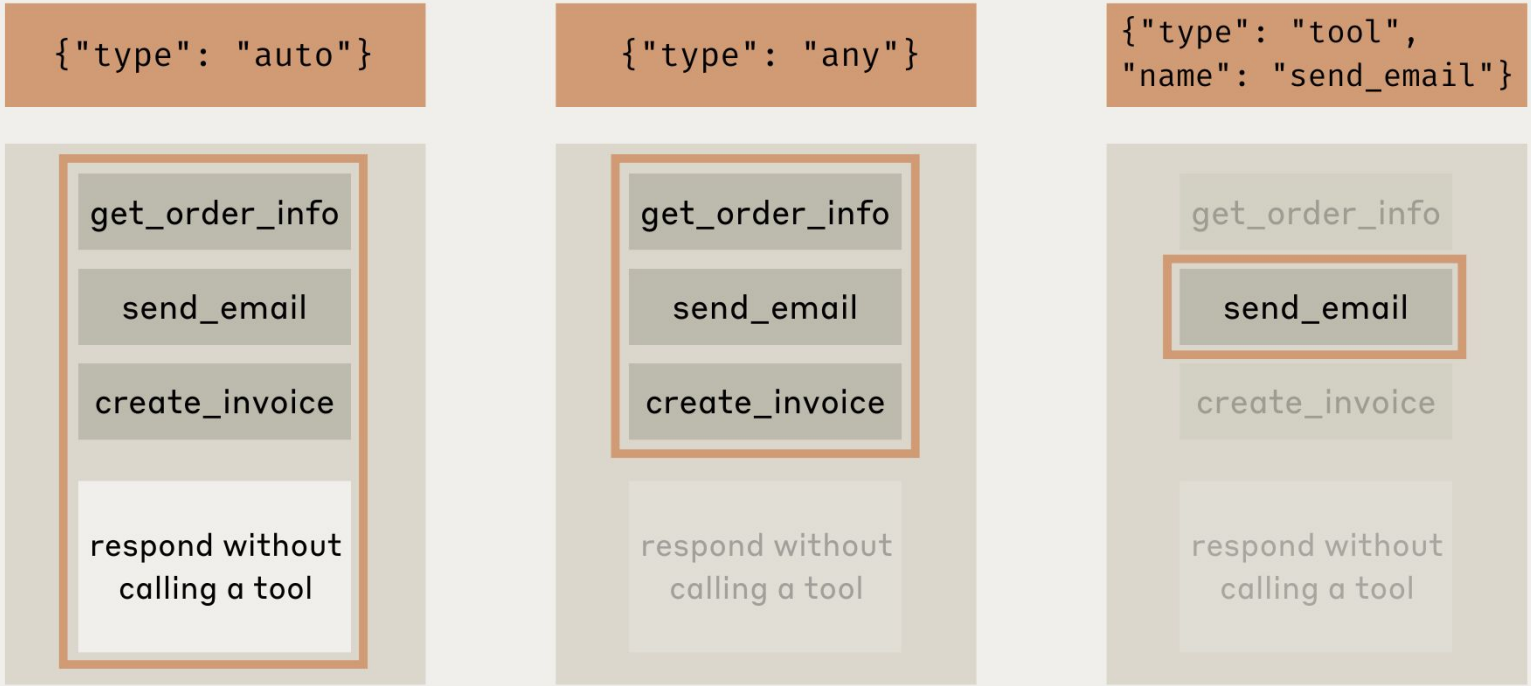
The Multi-Turn Tool Pattern



Controlling Tool usages

- We can control how Claude use tools using the `tool_choice` parameter. There are three options:
  - `auto` allows Claude to decide whether to call any provided tools or not. This is the default value.
  - `any` tells Claude that it must use one of the provided tools, but doesn't force a particular tool.
  - `tool` allows us to force Claude to always use a particular tool.
- **Note** that when you have `tool_choice` as `any` or `tool`, **they will prefill** the assistant message to force a tool to be used.
  - Won't be able to CoT before finishing `tool_use` content blocks.

Tool choice


{"type": "auto"}
get_order_info
send_email
create_invoice
respond without calling a tool
{"type": "any"}
get_order_info
send_email
create_invoice
respond without calling a tool
{"type": "tool", "name": "send_email"}
get_order_info
send_email
create_invoice
respond without calling a tool

OpenAI’s Tool choice

- By default the model will determine when and how many tools to use. You can force specific behavior with the tool_choice parameter.
  - **Auto**: (Default) Call zero, one, or multiple functions.
    `tool_choice: "auto"`
  - **Required**: Call one or more functions.
    `tool_choice: "required"`
  - **Forced Function**: Call exactly one specific function.
    `tool_choice: {"type": "function", "name": "get_weather"}`
  - **None**: Call no function.
    `tool_choice: "none"`

Auto	Required	Forced Function
“auto”	“required”	{“type: “function”, “function”: {“name”: “schedule_event”}}
get_weather()	get_weather()	get_weather()
schedule_event()	schedule_event()	schedule_event()
send_email()	send_email()	send_email()
no function calls	no function calls	no function calls

CHA

10th ANNIVERSARY

Prompt engineering your tools

Tool definitions and specifications should be given just as much prompt engineering attention as your overall prompts.

- **Put yourself in the model's shoes**. Is it obvious how to use this tool, based on the description and parameters, or would you need to think carefully about it? If so, then it’s probably also true for the model.
  - A good tool definition often includes example usage, edge cases, input format requirements, and clear boundaries from other tools.
- **Test how the model uses your tools**.

Best practices for defining functions

Apply software engineering best practices

- **Make the functions obvious and intuitive**. (principle of least surprise)
- **Use enums and object structure to make invalid states unrepresentable**. (e.g. `toggle_light(on: bool, off: bool)` allows for invalid calls)
- **Pass the intern test**. Can an intern/human correctly use the function given nothing but what you gave the model? (If not, what questions do they ask you? Add the answers to the prompt.)
- **Poka-yoke your tools**. Change the arguments so that it is harder to make mistakes.

https://platform.openai.com/docs/guides/function-calling?api-mode=res

Best practices for defining functions

Offload the burden from the model and use code where possible.

- **Don't make the model fill arguments you already know**. For example, if you already have an `order_id` based on a previous menu, don't have an `order_id` param – instead, have no params `submit_refund()` and pass the `order_id` with code.
- **Combine functions that are always called in sequence**. For example, if you always call `mark_location()` after `query_location()`, just move the marking logic into the query function call.

Best practices for defining functions

Keep the number of functions small for higher accuracy.

- **Evaluate your performance** with different numbers of functions.
- **Aim for fewer than 20 functions** at any one time, though this is just a soft suggestion.

Summary

- API workflow
- Prompt Engineering
- Tool Uses